

编译原理习题课



Exercise 3.4

Given the following regular expression

$$1^*(0 \mid 01)^*$$

(1) Transform it to an equivalent finite automaton.

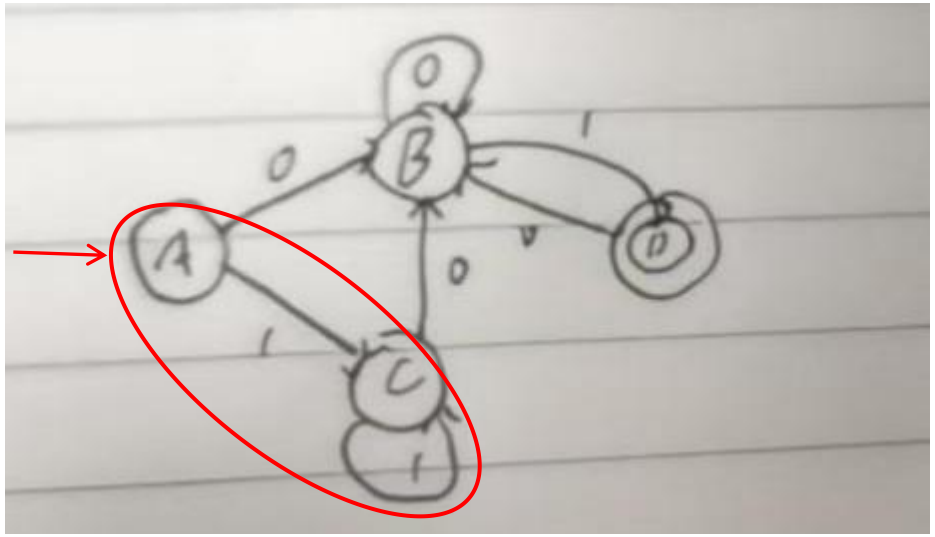
(2) Construct an equivalent DFA for the result of exercise (1).

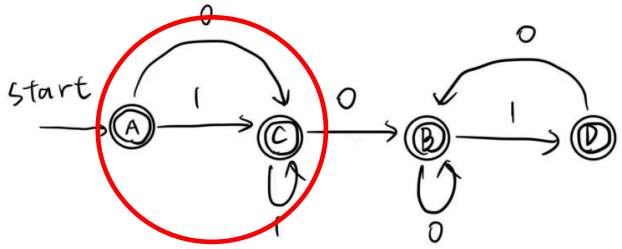
(3) Reduce the result of (2) and get a reduced DFA.

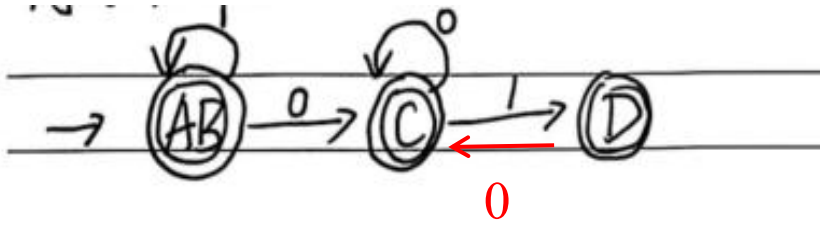
Exercise 3.4

解题思路：考虑正则表达式无法生成的式子，可知当出现0之后，后面出现的1不能连续出现，据此可以用于测试构建的NFA和DFA是否正确。

常见错误：
1.没画图
2.没有化简到最简
3.化简错误







Exercise 3.5

Given the alphabet Σ= { z, o, / }, a comment in a program over begins with "/o" and ends with "o/". Embedded comments are not permitted.

(1) Draw a DFA that recognizes nothing but all the comments in the source programs.

(2) Write a single regular expression that exactly describes all the comments in the source programs.

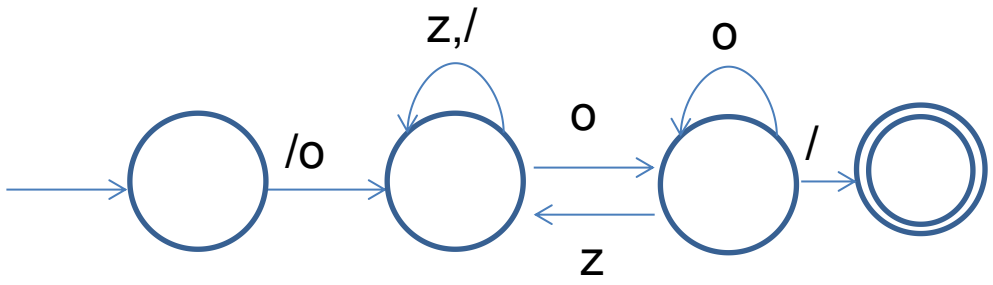
Exercise 3.5

题目解析：该问题需要构造一个类似“/o.....o/”的正则表达式。在做之前需要考虑清楚题目中一些特殊情况，如嵌套语法是不被允许的，即“/o../o..o/..o/”是不正确的，但是“/o../o..o/”是合理的，因此，在开始前我们提前写出一些测试用例用于验证我们最终答案是否正确

错误用例：“/o..o/..”

正确用例：“/oz/oz//o/”等

正确答案：



reduced DFA

/o(z | / | o*z)*oo*/

答案不唯一

1. /o(/* o* z)*/*o+/
2. /o/*(o* z /*)*o+/
3. /o(o* z | /)*o+/

其余正确答案

Exercise 3.5

常见错误答案：

1. /o(o | z | /)*o/　　（匹配过多的串，提前出现o/）
2. /o (/ | oz | oo)*o+/　　（提前出现了o/）
3. /o(z |/ | o(z|o))*o/　（提前出现了o/）
4. /o (z ｜ / ｜ o+z)*o*/　（最后的可能会出现/o/的错误情况）

Exercise 4.2

Consider the context-free grammar

$$S \rightarrow a\ S\ b\ S \mid b\ S\ a\ S \mid \varepsilon$$

Can you construct a predictive parser for the grammar? and why?

题目解析：能不能构造predictive parser，则需要判断该文法是否具有二义性。

第一种方法，计算FIRST（S）和FOLLOW（S)，若FIRST（S）中有ε，则计算FIRST（S）和FOLLOW（S）的交集是否为空，若不为空，则具有二义性。

第二种方法，举出一个例子，如abab，给出两种构造方法，用于判断二义性。

Exercise 4.2

常见错误答案：

1. 仅做出判断，不解释

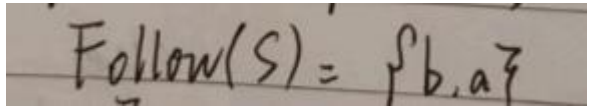
- 考虑上下文无关文法

$$S \to aSbS \mid bSaS \mid \varepsilon$$

- 你能为该文法构造一个预测解析器吗？为什么？

解答：不能，因为该文法有二义性。

2. FOLLOW内不包含结束符$



Exercise 5.1

Given the following grammar

$$S \rightarrow (L) \mid a$$

$$L \rightarrow L , S \mid S$$

1.Construct an LL(1) parsing table for the grammar .

Note: you must eliminate the left recursion first.

2.Draw the detailed process of the parsing of the sentence (a, (a, a)), follow the style in the previous slides.

Exercise 5.1

题目解析：构建LL（1）语法表步骤

1.消除左递归，构建新的文法

2.求FIRST，FOLLOW

3.根据FIRST、FOLLOW，构建LL（1）语法表

4.做预测分析过程

常见错误：

1.未构建LL（1）语法表，未计算FIRST和FOLLOW

Exercise 5.2

Given the following grammar

$$A \rightarrow B \mid B\ C$$
$$B \rightarrow a\ B \mid \varepsilon$$
$$C \rightarrow a\ b$$

1.Left factor the grammar.

2.After left factoring, is the grammar an LL(1) grammar? or is it an LL(k) grammar? and why?

Note: you may try the input string ab.

Exercise 5.2

题目解析：LL（k）：在自左向右扫描输入、构造最左推导时，只需要向前看 k 个输入符号，就能够唯一决定当前应该使用哪一个产生式。

第一问提取左因子后：

$$A \rightarrow B\ A'$$
$$A' \rightarrow C \mid \varepsilon$$
$$B \rightarrow a\ B \mid \varepsilon$$
$$C \rightarrow a\ b$$

第二问：由$B \rightarrow a\ B \mid \varepsilon$，中，FIRST（aB）$=\{a\} \cap FOLLOW(B)=\{a, \$\}$并不为空，因此，不是LL（1）。

考虑向前看2个，若为aa，则可以执行$B \rightarrow aB$，若为ab，则可以执行$B \rightarrow \varepsilon$,$A' \rightarrow C$,$C \rightarrow a\ b$,若为a$,则可以执行$A' \rightarrow \varepsilon$，因此为LL（2）

Exercise 5.2

常见错误：

1.第二问对原文法进行判断处理，导致得到不是LL（k）文法的错误结论。

2.做出了不是LL（1）文法的判断但是没有进行进一步的分析。

Exercise 6.1

	a	(	)	,	$
a			≻	≻	≻
(	≺	≺	≡	≺	
)			≻	≻	≻
,	≺	≺	≻	≻	
$	≺	≺			

Consider the grammar

$S \rightarrow (L) \mid a$

$L \rightarrow L , S \mid S$

We have the following operator-precedence relations for the grammar.

Show the detailed process of the parsing of the sentence (a, (a, a)), follow the style in the previous slides.

Exercise 6.1

题目解析：根据文法和运算符优先关系，按照ppt的格式一步步构造分析表即可

Step	Stack	Input	Reference	Action	Output
1	$	(a,(a,a))$	$<(	shift	
2	$(	a,(a,a))$	(<a	shift	
3	$(a	,(a,a))$	a>,	reduce	S→ a
...	...	...	...	...	...

$$S \rightarrow (L) \mid a$$
$$L \rightarrow L , S \mid S$$

	a	(	)	,	$
a			≻	≻	≻
(	≺	≺	≡	≺	
)			≻	≻	≻
,	≺	≺	≻	≻	
$	≺	≺			

常见错误：

1.分析过程不规范，未形成表格或表格内容不规范

LR 分析核心定义

概念	定义
LR(0) item	形如 A → α · β，点表示已经识别到的位置
closure(I)	对项目集I，若[A→α·Bβ]在I中，且B→ γ是文法产生式，则将B→ ·γ加入I中。重复该过程，直到不能再加入新项目为止。
GOTO(I, X)	对项目集I和文法符号X，若 A→α· X β∈I，则将圆点越过X，得到 A→αX· β。所有此类项目构成集合后，再求closure，所得集合即为 GOTO(I,X)。
SLR(1)	SLR(1) 分析表基于 LR(0) 项集族构造。若某状态中含有完成项目A→α· ，则只在 FOLLOW(A) 中的终结符列填入归约动作 r:A→α。若构造出的 ACTION/GOTO 表无 shift/reduce 或 reduce/reduce 冲突，则该文法为 SLR(1) 文法。
LR(1)	LR(1) 项写作 [A→α· β,a]，其中 a 是lookahead符号，表示当已经识别 α，接下来希望识别 β，且归约后可能遇到输入符号 a。LR(1) 归约动作只在该项目自身的 lookahead 符号a上填写，而不是在整个FOLLOW(A) 上填写。

判断是否 LR：不是看“有没有左递归”，而是看 ACTION 表同一格是否出现多重动作。

Exercise 7.1

Consider the grammar

$S \rightarrow (SR \mid a$
$R \rightarrow ,SR \mid)$

Try to construct an SLR(1) parsing table for the grammar, and see if there are conflicts in the parsing table.

Exercise 7.1 题目解析

增广文法：S' → S；产生式编号 1:S→(SR，2:S→a，3:R→,SR，4:R→)。
FIRST(S)={ (, a }，FIRST(R)={ ,,) }；FOLLOW(S)=FOLLOW(R)={ ,,), $ }。

ACTION 表

State	(	a	,	)	$
0	s2	s3			
1					acc
2	s2	s3			
3			r2	r2	r2
4			s6	s7	
5			r1	r1	r1
6	s2	s3			
7			r4	r4	r4
8			s6	s7	
9			r3	r3	r3

GOTO 表

State	S	R
0	1	
1		
2	4	
3		
4		5
5		
6	8	
7		
8		9
9		

Exercise 7.1 常见错误

错误类型	具体错误
集合误算	FOLLOW(S) 或 FOLLOW(R) 漏掉 {$,)} ， 导致归约列错误。
表格不一致	项集编号、GOTO 边、ACTION / GOTO 表编号对不上。

Exercise 7.2

Consider the grammar

$S \rightarrow Sab \mid aR$
$R \rightarrow S \mid a$

Is the grammar an SLR(1) grammar? and why?

Exercise 7.2 题目解析

判断 SLR(1) 不能只看左递归；关键是找 ACTION 表同一格的多重动作。

项目	含义	在输入 a 上的动作
R → S ·	完成项目；按 FOLLOW(R) 归约	reduce R→S
S → S · a b	点后为终结符 a	shift a

集合	结果	说明
FOLLOW(S)	{ a, $ }	S 为开始符号；S→S a b 中 S 后接 a
FOLLOW(R)	{ a, $ }	S→aR 中 R 在末尾，继承 FOLLOW(S)

结论：同一状态在输入 a 上同时需要 shift 与 reduce，出现 shift/reduce conflict，所以该文法不是 SLR(1)。

Exercise 7.2 常见错误

常见错误	为什么不充分	规范写法
只写“不是 SLR(1)”	没有可复核的冲突证据	写出状态中的两个项目和冲突输入 a
把 FIRST 当作归约依据	SLR 归约依据是 FOLLOW(左部)	先计算 FOLLOW(S), FOLLOW(R)
说“有左递归所以不是”	LR 分析允许左递归	冲突来自 ACTION 表同格多动作

Exercise 7.3

Consider the grammar

$S \rightarrow A$
$A \rightarrow BA \mid \varepsilon$
$B \rightarrow aB \mid b$

Prove that the grammar is an LR(1) grammar.
Construct an LR(1) parsing table for the grammar.
Show the detailed parsing procedure for the sentence abab, following the style in slides of this lecture.

Exercise 7.3 LR(1) 分析表

产生式编号：1:S→A；2:A→BA；3:A→ε；4:B→aB；5:B→b。

ACTION 表

State	a	b	$
0	s4	s5	r3
1			r1
2	s4	s5	r3
3			acc
4	s4	s5	
5	r5	r5	r5
6			r2
7	r4	r4	r4

GOTO 表

State	S	A	B
0	3	1	2
1			
2		6	2
3			
4			7
5			
6			
7			

所有 ACTION 表项唯一：无 shift/reduce 或 reduce/reduce conflict，因此该文法是 LR(1)。

Exercise 7.3：abab 分析过程（1/2）

LR(1)

Step	Stack	State	Input	Action	Output
1	$	0	abab$	shift a → state 4	
2	$ a	0 4	bab$	shift b → state 5	
3	$ a b	0 4 5	ab$	reduce B→b (r5); goto(4,B)=7	B→b
4	$ a B	0 4 7	ab$	reduce B→aB (r4); goto(0,B)=2	B→aB
5	$ B	0 2	ab$	shift a → state 4	
6	$ B a	0 2 4	b$	shift b → state 5	
7	$ B a b	0 2 4 5	$	reduce B→b (r5); goto(4,B)=7	B→b

说明：每一行只执行一个 LR 动作；归约后的 GOTO 写在同一行，用于说明新状态来源。

Exercise 7.3：abab 分析过程（2/2）

LR(1)

Step	Stack	State	Input	Action	Output
8	$ B a B	0 2 4 7	$	reduce B→aB (r4); goto(2,B)=2	B→aB
9	$ B B	0 2 2	$	reduce A→ε (r3); goto(2,A)=6	A→ε
10	$ B B A	0 2 2 6	$	reduce A→BA (r2); goto(2,A)=6	A→BA
11	$ B A	0 2 6	$	reduce A→BA (r2); goto(0,A)=1	A→BA
12	$ A	0 1	$	reduce S→A (r1); goto(0,S)=3	S→A
13	$ S	0 3	$	accept	

说明：每一行只执行一个 LR 动作；归约后的 GOTO 写在同一行，用于说明新状态来源。

Exercise 7.3 常见错误

LR(1)

常见错误	错误原因	改正建议
把 LR(1) 项写成 LR(0) 项	无法证明无冲突	项目必须写成 [A→α · β, a]
A→ε 在所有终结符上归约	会错误制造冲突	只在该 LR(1) 项的 lookahead 上归约
abab 分析漏掉连续归约	栈无法回到 S	注意最后两次 A→BA
用“无二义性”替代证明	不是 LR(1) 的构造证明	以分析表无冲突作为证明

Exercise 7.4*

(DBv2, pp.278, ex.4.7.4) Show that the grammar

$S \rightarrow Aa \mid bAc \mid dc \mid bda$
$A \rightarrow d$

is LALR(1) but not SLR(1).

Exercise 7.4* 题目解析

LALR(1)

目标	关键状态 / 原因
not SLR(1)	LR(0) 状态含 S→d · c 与 A→d · ； FOLLOW(A)={a,c}，因此输入 c 上 s/r 冲突
LR(1) 可区分	A→d · 的 lookahead 可由上下文区分为 a 或 c，不必在所有 FOLLOW(A) 列归约
LALR(1)	合并同心 LR(1) 项后不产生新的冲突，因此仍可用 LALR(1) 表分析

Exercise 7.4* 第一步：证明 not SLR(1)

FOLLOW(A) = { a, c }。因此 SLR 中 A→d· 要在 a 与 c 两列都归约。

LR(0) 状态	状态内关键项目	冲突输入	冲突类型
读入 d 后	S → d · c A → d ·	c	shift c 与 reduce A→d
读入 b d 后	S → b d · a A → d ·	a	shift a 与 reduce A→d

结论：SLR 使用 FOLLOW(A) 统一填归约列，导致至少两个 shift/reduce conflict，因此不是 SLR(1)。

Exercise 7.4* 第二步：LR(1) 消除冲突

状态 I4
[A → d ·, a]
[S → d · c, $]

在 c 上 shift；只在 a 上 reduce

对比

状态 I7
[A → d ·, c]
[S → b d · a, $]

在 a 上 shift；只在 c 上 reduce

输入符号	SLR 处理	LR(1) 处理
c	A→d· 会在 c 上归约，与 S→d·c 移进冲突	状态 I4 中 A→d· 的 lookahead 是 a，不在 c 上归约
a	A→d· 会在 a 上归约，与 S→bd·a 移进冲突	状态 I7 中 A→d· 的 lookahead 是 c，不在 a 上归约

Exercise 7.4* 第三步：为什么仍是 LALR(1)

LALR(1) 的构造：将 canonical LR(1) 自动机中 LR(0) core 相同的状态合并，并合并 lookahead。

观察项	本题结论
关键状态 I4 的 core	{ A→d·, S→d·c }
关键状态 I7 的 core	{ A→d·, S→bd·a }
是否会互相合并	不会，因为 LR(0) core 不同
合并后是否出现新冲突	不会；归约 lookahead 不会被错误扩展到移进列

最终结论：该文法不是 SLR(1)，但 canonical LR(1) 无冲突；合并同项后仍无冲突，因此是 LALR(1)。

Exercise 8.1

Given the translation scheme for the EQN language (see pp.40 in this lecture), calculate the height and depth of the input: **text sub text sub text**.

Suppose that for each **text**,
getHeight(ps, **text**.lexval) = 8 * ps
getDepth(ps, **text**.lexval) = 0

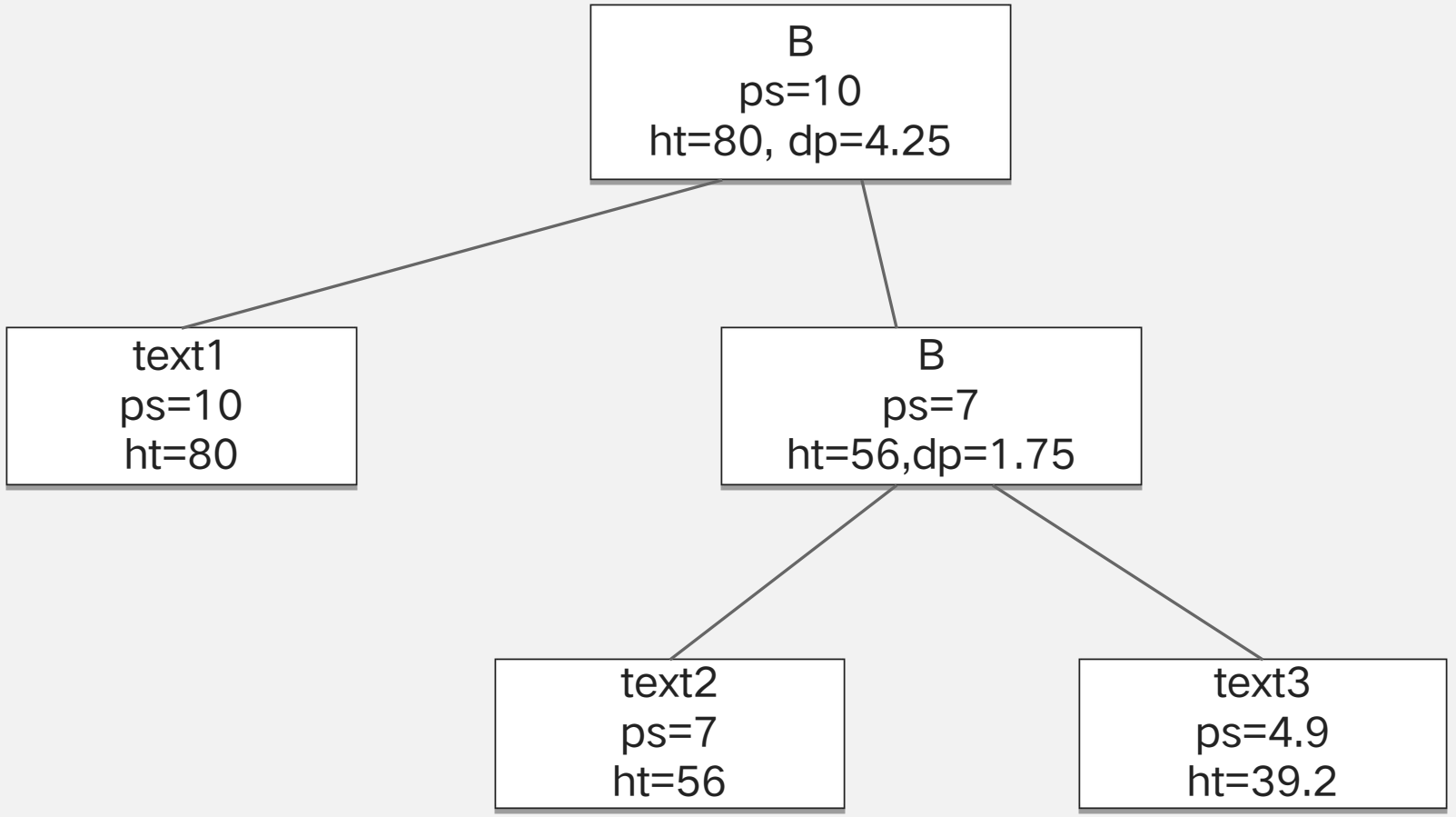
No.	Productions	Semantic Actions
1	$S \rightarrow B$	$B.ps = 10$
2	$B \rightarrow B_1\ B_2$	$B_1.ps = B.ps$ $B_2.ps = B.ps$ $B.ht = \max(B_1.ht, B_2.ht)$ $B.dp = \max(B_1.dp, B_2.dp)$
3	$B \rightarrow B_1$ **sub** B_2	$B_1.ps = B.ps$ $B_2.ps = B.ps \times 70\%$ $B.ht = \max(B_1.ht, B_2.ht - B.ps \times 25\%)$ $B.dp = \max(B_1.dp, B_2.dp + B.ps \times 25\%)$
4	$B \rightarrow (\ B_1\ )$	$B_1.ps = B.ps$ $B.ht = B_1.ht$ $B.dp = B_1.dp$
5	$B \rightarrow$ **text**	B.ht = getHight(B.ps, **text**.lexval) B.dp = getDepth(B.ps, **text**.lexval)

ps = point size
ht = height
dp = depth

Higher precedence, right associativity

课件说明：sub 具有更高优先级，并按右结合处理。

Exercise 8.1 属性传递图



右结合：text1 sub (text2 sub text3)；字号沿右操作数逐层乘 70%。

Exercise 8.1

课件说明：sub 具有更高优先级，并按右结合处理。

初始字号	S→B 时 B.ps = 10
继承属性	B→B1 sub B2：B1.ps=B.ps，B2.ps=B.ps×70%
综合属性	B.ht=max(B1.ht, B2.ht−B.ps×25%) B.dp=max(B1.dp, B2.dp+B.ps×25%)

节点	ps	ht	dp
text1	10	80	0
text2	7	56	0
text3	4.9	39.2	0

Exercise 8.1 题目解析

计算对象	公式	结果
内层 B23.ht	max(56, 39.2 − 7×25%)	56
内层 B23.dp	max(0, 0 + 7×25%)	1.75
根 B.ht	max(80, 56 − 10×25%)	80
根 B.dp	max(0, 1.75 + 10×25%)	4.25

最终答案：height = 80，depth = 4.25。

第 8 次作业常见错误

常见错误	错误原因
把 sub 当作左结合	少数答案语法树方向不清
text3.ps 写成 7	继承属性传递少一步
只给符号公式	没有代入初始 ps=10
漏写 depth	height 普遍较好，dp 更易漏

Exercise 9.1

Let A be declared as a [2..4] x [1..5] array
of integers and each integer occupy 4 bytes.
What is the translation result of input token string:

x := A[3, 2]?

Tips: use the translation scheme for Pascal.

DBv1, pp.484-485

Exercise 9.2

Let a be declared as a 5 x 6 array of integers
and each integer occupy 4 bytes.
What is the translation result of input token string:

i = a[3][2]?

Tips: use the translation scheme for C/C++.

DBv2, pp.383

Exercise 9.1 / 9.2 地址翻译对比

维度	Pascal: A[3,2]	C/C++: a[3][2]
下界	low1=2, low2=1	默认从 0 开始
行宽	n2=5，w=4	n2=6，w=4，因此一行 24 bytes
核心公式	V=i1×n2+i2；C=low1×n2+low2；offset=(V−C)×w	offset=i1×(n2×w)+i2×w
本题偏移	(3×5+2 − 11)×4 = 24 bytes	3×24+2×4 = 80 bytes

Exercise 9.3

What is the translation result of input token string:

x < 100 || x > 200 && x != y?

Tips: use the translation scheme for boolean expressions with backpatching.
Suppose that the start position of the generated code is 100.

DBv2, pp.411-416

Exercise 9.3 题目解析

Backpatch

优先级：x < 100 || (x > 200 && x != y)。短路求值通过 trueList / falseList 回填实现。

```
100: if x < 100 goto _T_
101: goto _102_
102: if x > 200 goto _104_
103: goto _F_
104: if x != y goto _T_
105: goto _F_
```

回填操作	含义
backpatch({101}, 102)	|| 左操作数为假时，进入右操作数
backpatch({102}, 104)	&& 左操作数为真时，进入右操作数
trueList = {100,104}	真出口等待外层语句回填
falseList = {103,105}	假出口等待外层语句回填

第 9 次作业常见错误

类别	常见错误
数组寻址	Pascal 题直接用 0 下界，或把 A[3,2] 误算为 A[1,2]
数组寻址	只给偏移，不计算具体的byte
布尔回填	&& 和 || 优先级混淆

Quiz#5 题目

中山大學 SUN YAT-SEN UNIVERSITY

Item #5.1 给定文法

$$S \rightarrow B$$
$$B \rightarrow BE+ \mid -E$$
$$E \rightarrow e \mid Ee \mid \varepsilon$$

(1) 写出该文法的拓广文法，并为新文法的每一产生式加上编号（第一个产生式从 0 开始编号）；再用一句话解释在构造 LR 分析表时拓广文法的原因。

(2) 求出所有非终结符号的 FIRST 集以备后用。

(3) 根据 LR(1)有效项目集族画出识别该文法所有活前缀的 DFA。

(4) 判断该文法是否一个 LR(1)文法，并说明你的理由。

Quiz#5 解析

Item #5.1 〖参考答案〗

(1) 拓广文法：

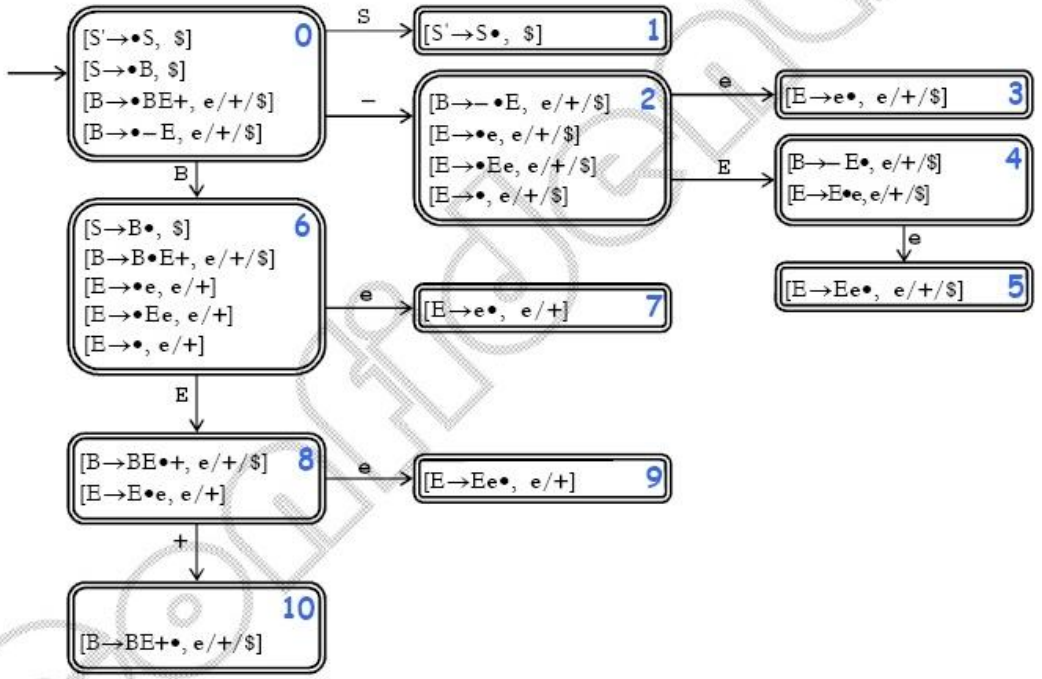
(0) S' → S
(1) S → B
(2) B → BE+
(3) B → −E
(4) E → e
(5) E → Ee
(6) E → ε

解释：在构造 LR 分析表时拓广文法可以避免文法开始符号出现在产生式右边，从而接受项目是惟一的。

(2) 求 FIRST 集：

FIRST(S) = FIRST(B) = { − }
FIRST(E) = { e , ε }

(3) 构造识别活前缀的 DFA：



(4) 该文法不是一个 LR(1)文法；因为在状态 2（或答状态 4、6）有“移进－归约”冲突，当向前看符号为 e 时产生冲突。

常见错误：

- 未能正确解释拓广文法原因。
- 画 DFA 时向前看符号错。

Quiz#5 题目

Item #5.2 给定语法制导定义（Syntax-Directed Definition）如下：

产生式	语义动作
S → A B C D	x = 10 * x + 1;
D → **d**	x = x + 5;
C → **c c**	x = 5 * x + 1;
B → B **b**	x = 3 * x + 1;
B → **b**	x = x + 1;
A → **e** B **a**	x = 2 * x + 1;

对于输入句子 **ebbabbccd**，如果在自底向上 LR 分析过程的同时执行语义动作，则上述语义动作将执行 _____ 次；假设 x 的初始值为 0，则语义动作执行完后 x 的最终值为 _____。

Quiz#5 解析

【参考答案】8 次（即推导或归约的次数）；x 最终值为 1611。

在 LR 分析过程中执行语义动作的过程如下：

句型	所使用的产生式	计算
S	S → A B C D	10 * 161+ 1 = 1611
ABCD	D → **d**	156 + 5= 161
ABC**d**	C → **c c**	5 * 31 + 1= 156
AB**ccd**	B → B **b**	3 * 10 + 1= 31
AB**bccd**	B → **b**	9 + 1= 10
A**bbccd**	A → **e** B **a**	2 * 4 + 1= 9
eB**abbccd**	B → B **b**	3 * 1 + 1= 4
eB**babbccd**	B → **b**	0 + 1 = 1
ebbabbccd		

Quiz#5 题目

Item #5.3　设有如下语言：

```
Expr  →  for id := int to int do Expr
      |  id := Expr
      |  Expr ; Expr
      |  Expr * Expr
      |  Expr + Expr
      |  id
      |  int
```

定义一个表达式的开销，是其所有子表达式的开销、再加上表达式本身计算的开销。例如，表达式 **a** **+** E的开销是1加上子表达式 **a** 的开销和子表达式 E 的开销，也可简单地说加法的额外开销是1；类似地，定义乘法的额外开销是2，顺序操作的额外开销是0。赋值的额外开销是1；除额外开销之外，赋值的开销还包括作为右值的子表达式的开销，但不包括作为左值的变量的开销。循环的开销是3再加上循环中每一次迭代的子表达式的开销（即子表达式开销乘以循环次数再加3），每一个变量 **id** 与常量 **int** 作为表达式的开销均为1。

给出一个计算上述开销函数定义的语法制导定义（Syntax-Directed Definition，SDD）。

你可假设有一个属性 val 包含单词的词法分析值；此外，你也可按自己的需要定义其他属性。

Quiz#5 解析

Item #5.3【参考答案】注意这一类题目通常存在多种合理的答案。

计算表达式开销的语法制导定义：

语法规则	语义规则
Expr → **for** **id** := int_1 **to** int_2 **do** $Expr_1$	**if** (int_2.val >= int_1.val) **then** Expr.cp := (int_2.val - int_1.val + 1) * $Expr_1$.cp + 3; **else** Expr.cp := 3; **endif**;
Expr → **id** := $Expr_1$	Expr.cp := $Expr_1$.cp + 1;
Expr → $Expr_1$; $Expr_2$	Expr.cp := $Expr_1$.cp + $Expr_2$.cp;
Expr → $Expr_1$ * $Expr_2$	Expr.cp := $Expr_1$.cp + $Expr_2$.cp + 2;
Expr → $Expr_1$ + $Expr_2$	Expr.cp := $Expr_1$.cp + $Expr_2$.cp + 1;
Expr → **id**	Expr.cp := 1;
Expr → **int**	Expr.cp := 1;

Quiz#4 题目

考虑如下的上下文无关文法（Context-Free Grammar）：

S → A

A → A + A | B + +

B → y

其中，每一个“+”是一个单独的符号（即单词，token）。由于开始符号（Start Symbol）仅有一个产生式，故该文法无需进行拓广（augment），S可直接作为LR分析器的开始符号。

1、 画出输入串 y + + + y + + 的分析树（Parse Tree）。

2、 记LR(1)有效项目（Valid Item）I = [S → • A, $]，求closure({I})。提示：即求识别该文法所有活前缀（Viable Prefix）的DFA的开始状态中包含的所有LR(1)有效项目。

3、 在构造识别该文法的所有活前缀的确定有限自动机（DFA）时，有一个状态 Q = {

[A → A + A •, + / $],

[A → A • + A, + / $]

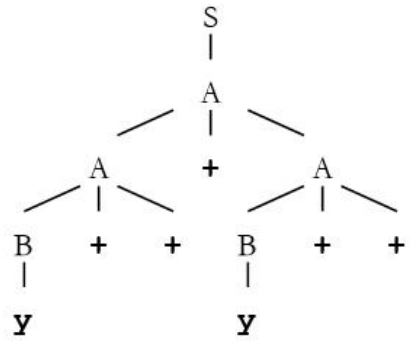
}，求 go(Q, +)。提示：即求从状态Q出发，经过有向边+到达的状态；列出该状态中的所有LR(1)有效项目。

4、 该文法存在“移进－归约”冲突（Shift-Reduce Conflict）。请给出上述DFA中存在这种冲突的状态（即列出状态中所有的LR(1)有效项目），并指出在处理哪一个输入符号时会发生冲突?

Quiz#4 解析

【参考答案】

1、分析树如下：



2、closure({ [S → • A, $] }) = {

[S → • A, $],
[A → • A + A, + / $],
[A → • B + +, + / $],
[B → • y, +] }

3、go(Q, +) = closure({ [A → A + • A, + / $] }) = {

[A → A + • A, + / $],
[A → • A + A, + / $],
[A → • B + +, + / $],
[B → • y, +] }

4、在包含项目[A → A + A •, + / $]和[A → A • + A, + / $]的那个状态、遇到输入符号+时，会发生“移进－归约”冲突。

Quiz#3 题目

考虑两个 LL(1)文法，每个都只有一个非终结符号（设第一个文法的非终结符号为 A，第二个文法的非终结符号为 B），且不含ε产生式。考察第三个文法，该文法的开始符号为 S，并且产生式通过合并上述两个文法的所有产生式得到，然后再添加以下产生式：

$$S \rightarrow A \mid B$$

请回答以下问题并解释理由：

1 在第三个文法中，是否可能从文法的开始符号 S 推导出空串？为什么？

2 为保证第三个文法是 LL(1)的，原先的两个文法必须满足的充分必要条件是什么？请不要给出 A 或 B 的具体例子，你需要描述的是针对上述形式的所有文法均须满足的充要条件。

3 为保证第三个文法是无二义的，原先的两个文法必须满足的充分必要条件是什么？请不要给出 A 或 B 的具体例子，你需要描述的是针对上述形式的所有文法均须满足的充要条件。

Quiz#3 解析

〖参考答案〗

1　不可能，因为从 A 或 B 均不可能推导出空串。

2　充要条件：A 和 B 能够推出的首字符不可相同，即 FIRST(A) ∩ FIRST(B) = ∅。注意，题目已指出这两个文法是 LL(1)的，且 A 或 B 均不可能推导出空串。

3　充要条件：这两个文法产生的语言的交集为空，即 L(A) ∩ L(B) = ∅，其中 L(X)表示由文法 X 产生的语言。注意，由于题目已指出这两个文法是 LL(1)的，故它们一定是无二义的。

Quiz#3 题目

考虑下面文法 G，

$$S \rightarrow Q\ \mathbf{a}$$

$$Q \rightarrow Q\ \mathbf{b} \mid \mathbf{c} \mid \varepsilon$$

1 先消除文法中的直接左递归，得到新文法 G'。

2 再计算新文法 G'中所有非终结符（Nonterminal）的 FIRST 集和 FOLLOW 集。

3 最后构造 G'的 LL(1)分析表。

Quiz#3 解析

【参考答案】

1 消除左递归，得到文法 G'：

S → Q **a**

Q → **c** Q' | Q'

Q' → **b** Q' | **ε**

2 求非终结符的 FIRST 集和 FOLLOW 集：

FIRST(S) = { **a**, **b**, **c** }, FIRST(Q) = { **b**, **c**, **ε** }, FIRST(Q') = { **b**, **ε** }

FOLLOW(S) = { **$** }, FOLLOW(Q) = FOLLOW(Q') = { **a** }

3 最后构造 LL(1)分析表：

	a	**b**	**c**	**$**
S	Q **a**	Q **a**	Q **a**	
Q	Q'	Q'	**c** Q'	
Q'	**ε**	**b** Q'		

Quiz#2 题目

An integer constant consisting of a sequence of digits is taken to be decimal (base ten) unless it begins with **0** (digit zero). A sequence of digits starting with **0** is taken to be an octal integer (base eight). The digits **8** and **9** are not octal digits. A sequence of digits preceded by **0x** or **0X** is taken to be a hexadecimal integer (base sixteen). The hexadecimal digits include **a** or **A** through **f** or **F**…. The type of an integer depends on its … suffix. [An integer may have no suffix.] …If [the integer] if suffixed by **u** or **U**, its type is …. If it is suffixed by **l** or **L**, its type is …. [Either or both suffixes may appear at most once each in either order.]

试为C++语言的整型常量设计一个正规定义式，要求其中仅使用正则表达式的基本算子“|”、“*”和连接，禁止使用 lex 或 flex 等软件工具扩展的表示；在正则表达式中可以使用括号和空串ε。

Quiz#2 解析

〖参考答案〗

Octal → **0** | **1** | **2** | **3** | **4** | **5** | **6** | **7**

NonZero → **1** | **2** | **3** | **4** | **5** | **6** | **7** | **8** | **9**

Decimal → **0** | NonZero

Hex → Decimal | **a** | **A** | **b** | **B** | **c** | **C** | **d** | **D** | **e** | **E** | **f** | **F**

OptU → **u** | **U** | ε

OptL → **l** | **L** | ε

Suffix → OptU OptL | OptL OptU

Number → **0** Octal* | NonZero Decimal* | (**0x** | **0X**) Hex Hex*

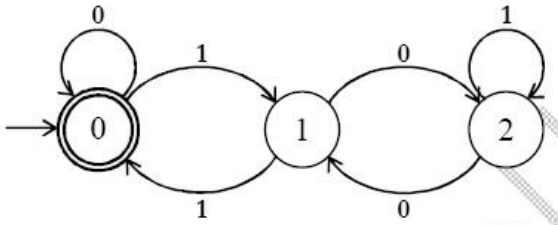
Integer → Number Suffix

Quiz#2 题目

构造一个 DFA，该 DFA 识别的语言是所有能被 3 整除的二进制数。

〖参考答案〗

解题关键：用 3 个状态分别表示输入串整除 3 的余数为 0、1 和 2；状态转移 $\delta(0, 1) = 1$ 相当于 $y = x * 2 + 1$，即 x 被 3 整除则有 y 的余数为 1。

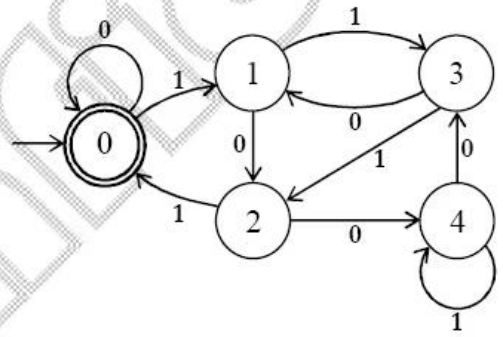


【备注】

1、注意题目要求画出 DFA，如果画成 NFA 则属基本概念理解不清。

2、识别被 3 整除后余 1 或余 2 的数：修改终态即可。

3、被 7 整除的数：状态数目变为 7。类似地，识别能被 5 整除的所有二进制数的 DFA 如下：



4、八进制或十进制数：每一状态的射出弧分别有 8 条或 10 条。

Quiz#2 题目

1、画出一个 DFA，该 DFA 恰好识别所有不含 **011** 子串的所有二进制串（注意不要 *under/over spec.*，下同）。

2、再画一个 DFA，该 DFA 恰好识别所有不含 **101** 子串的所有二进制串。

3、从上述两个例子中，总结出解决这一类问题的普遍规律。

谢谢大家

编译原理第二次习题课

